Week 6 Lab 2 Optional Challenge: Model an Additional Business Module

Before You Begin

Complete this challenge only after finishing all five core Lab 2 diagrams.

This is a design exercise. You do not need to implement the selected module.

Choose one capability:

```
Milestone Management
```

or:

```
Academic Supervisor Management
```

Your proposal must fit inside the existing six-project architecture and remain part of the same deployed ASP.NET Core application.

Select One Existing Use Case

For Milestone Management, use:

```
W5-UC02: View Enrolment Milestones
```

For Academic Supervisor Management, select one:

```
W4-UC04: Assign Academic Supervisor to Student
W4-UC05: View Student Supervisory Team
```

Use the existing use-case description. Do not invent a new user goal merely because the code structure changes.

Task 1: Propose the Folder and Namespace Structure

Do not create another set of .NET projects for the capability. Add folders and namespaces inside the existing projects.

Use the structure provided in lab 1 as reference and adapt it for Milestone and/or Academic Supervisor Management.

Document:

- Selected use case.
- Proposed folders and namespaces.
- Required ports.
- Expected persistence technology.
- Cross-capability collaboration.
- Key architectural decisions.

Task 2: Extend the C4 Component Diagram

Create a copy of the core Component diagram and add the proposed capability at component level.

Show its participation in:

- Domain.
- Business Use Cases.
- Relational Persistence Adapter.
- MVC and Razor Adapter.
- REST API Adapter where required.
- Application composition.
- SQLite.

Keep the diagram at component level. Do not fill it with individual controllers, interfaces, mapper classes, or `DbContext` classes. Put those details in the class diagram (in the next task).

If the targeted capability (e.g., milestone or academic supervisor) needs another capability's information (e.g., Student), show a deliberately exposed Business boundary or a consuming-use-case-owned output port.

Do not show direct access to another capability's:

- `DbContext`.
- Persistence record.
- Concrete gateway.

Task 3: Extend the Implementation Class Diagram

Add proposed classes for the selected use case.

Include where applicable:

- Domain entities and enumerations.
- Input-port interface and use-case implementation.
- Input and output DTOs.
- Business-owned output-port interface.
- Persistence records.
- Concrete EF gateway implementation.
- Read mapper required by the use case.
- MVC controller, view model, mapper, and Razor view.
- REST controller, request or response model, and API mapper.

- Application registration dependencies.

Mark proposed elements clearly:



Do not present proposed classes as implemented source code.

Retain only domain concepts supported by the existing requirements. Do not invent unsupported statuses, progression history, or relationships.

For a read-only use case, do not add write mappings such as `ToRecord` unless another proposed command requires them.

Task 4: Create One Proposed Sequence Diagram

Choose MVC or REST.

Show:

1. Actor or external client.
2. Incoming adapter.
3. Adapter-specific model.
4. Adapter mapper.
5. Business input DTO.
6. Business input port and use case.
7. Business-owned output port.
8. Concrete persistence adapter at runtime.
9. Persistence mapper.
10. SQLite.
11. Business output or outcome.
12. Adapter-specific response.
13. Relevant alternative flow.

For MVC, model the browser sending a request to the controller. Do not show the Razor view initiating the controller call.

For Milestone Management, decide explicitly who owns the Enrolment lookup port:

- A deliberately public Students Business contract, or
- A capability-specific output port owned by the Milestone use case.

Document the decision and avoid direct Students persistence access.

For Academic Supervisor Management, preserve the Student lookup boundary rather than accessing Students persistence directly.

Task 5: Check the Dependency Rule

Confirm:

- [ ] Domain additions depend on no outer project.
- [ ] Business additions depend only on Domain.
- [ ] Input and output ports belong to Business.
- [ ] Persistence implements Business-owned output ports.
- [ ] MVC and REST invoke Business input ports.
- [ ] Adapter projects do not reference one another.
- [ ] The capability does not access another adapter or `DbContext` directly.
- [ ] Application remains the only executable project.
- [ ] The capability remains part of the same deployment.

Deliverables

Retain:

```
week06-selected-module-structure.md
week06-selected-module-component-diagram.puml
week06-selected-module-class-diagram.puml
week06-selected-module-sequence.puml
```

Challenge Checklist

- [ ] I selected one existing use case.
- [ ] I proposed folders inside the existing six projects.
- [ ] My Component diagram remains at component level.
- [ ] Proposed classes are marked `<<proposed>>`.
- [ ] I introduced no unsupported domain assumptions.
- [ ] Business owns the required ports.
- [ ] Adapters do not depend on one another.
- [ ] Cross-capability access uses a deliberate Business boundary.
- [ ] My sequence shows runtime flow without reversing source dependencies.
- [ ] Application remains the only executable host.